UNIT 1, FOUNDATIONS OF AI AGENTS AND MICROSOFT FOUNDRY

LECTURE 2

Intelligent Agent Architectures

Five classical families, what each one is actually for,
and which one you already built without knowing it.

CSE476 | Rohit Bharti

SECTION 00

Your homework, first

Four minutes. Then we need better vocabulary.

HOMEWORK DEBRIEF

The products you found

Apply the four requirements from last time. Goal not question, tools, a loop with state, a way to stop.

MOST COMMON VERDICT

Fails the loop

It answers once. Very well, often. But it never looks at its own output and decides to try again.

SECOND MOST COMMON

Fails the goal

It accepts instructions, not outcomes. You are still doing the deciding and typing it in English.

THE HONEST ONES

Passes all four

Usually coding assistants and research tools. Notice these are also the expensive ones. That is not a coincidence.

BUT NOTICE THE PROBLEM WITH OUR VOCABULARY

Some of the products that passed all four still behave completely differently from each other. One remembers you, one starts fresh every time. One picks the first workable answer, one weighs options against each other. Our four requirements cannot tell those apart, and today we fix that.

Today

- Five classical agent architectures, in order of how much the agent knows and how far ahead it looks.
- PEAS, and why the environment decides the architecture rather than the other way round.
- Where a language model actually sits in this fifty year old picture.
- Build all four working architectures on one task, and read the difference in the traces.
- One thing modern LLM agents are genuinely not, despite what the marketing says.

HOLD ON TO THIS

This vocabulary is fifty years older than the models we use. It survived because it describes the shape of the problem, not the technology of the moment.

SECTION 01

Five architectures

Each one adds exactly one capability to the one before it.

The ladder. Each rung adds one thing.

01	**Simple reflex**	Sees now, acts now. No memory at all.	adds nothing, this is the floor
02	**Model based reflex**	Same, plus a memory of what it has already seen.	adds internal state
03	**Goal based**	Holds a desired end state and works out its own route.	adds foresight
04	**Utility based**	Several routes reach the goal. Picks the best one.	adds preference
05	**Learning**	Changes its own behaviour from experience.	adds improvement

Read the right hand column downwards. That is the entire lecture in five words.

THE FLOOR

One, simple reflex

THE LAYMAN VERSION

The thermostat

Too cold, heat on. Warm enough, heat off. It does not know it did the same thing an hour ago and it does not care what happens next.

THE TECHNICAL VERSION

Condition, action rules

A lookup from the current percept straight to an action. No state, no model of the world, no plan. Fast, cheap and completely predictable.

WHERE IT BREAKS

It fails the moment the right action depends on something not visible right now. Ask a reflex agent "what about the fifteenth" and it has no idea what you were talking about, because there is no it that remembers.

ADDS INTERNAL STATE

Two, model based reflex

THE LAYMAN VERSION

The security guard

Watches a corridor through one camera. Cannot see the whole building, but keeps a note of who walked in and has not walked out. The note is the difference.

THE TECHNICAL VERSION

A model of the world

Internal state that persists between percepts, updated by what it observes. It can now act on things it cannot currently see.

WHAT IT STILL CANNOT DO

It remembers, but it does not aim. There is still no notion of a desired end state, so it cannot work out a route to anything. It reacts, with a better picture of the world to react to.

ADDS FORESIGHT

Three, goal based

THE LAYMAN VERSION

The taxi driver

You say the station. Not which turns to take. The driver holds where you want to end up and works backwards from it, rerouting when a road is closed.

THE TECHNICAL VERSION

Search over future states

The agent holds a description of a desired state and evaluates candidate actions by whether they move it closer. Classically this is search or planning.

THIS IS THE JUMP THAT MATTERS

One and two both answer "what should I do now". Three answers "what should I do now, given where I am trying to end up". That second question is why the number of steps stops being fixed by you, which is exactly the Level 2 definition from last lecture.

ADDS PREFERENCE

Four, utility based

A goal tells you when to stop. It does not tell you which of three acceptable answers is best.

THE LAYMAN VERSION

The same taxi, better

Three routes reach the station on time. One is fastest, one is cheapest, one avoids the flyover you hate. A goal based driver takes any. A utility based one asks which you would prefer.

THE TECHNICAL VERSION

A utility function

A score over outcomes. The agent gathers what it needs, scores every viable option, and picks the maximum. Trade offs become explicit and therefore auditable.

WHY THIS IS THE ONE INDUSTRY ACTUALLY WANTS

Real requests are full of hidden preferences. Cheap but not too far. Fast but not risky. A goal based agent picks the first thing that works and cannot explain why. A utility based one shows its scores, and a decision you can see is a decision you can argue with.

ADDS IMPROVEMENT

Five, learning

PERFORMANCE ELEMENT	Does the actual job. This is any of the four architectures above.
CRITIC	Judges how well that went, against a fixed standard.
LEARNING ELEMENT	Changes the performance element based on what the critic said.
PROBLEM GENERATOR	Deliberately suggests unfamiliar actions, so the agent explores rather than repeating what already works.

THE HONEST NOTE, AND IT IS IMPORTANT

Almost nothing you will build this semester is a learning agent. The model weights do not change while your agent runs. It looks like learning because it carries a conversation, but memory within one session is not the same as changing behaviour from experience.

All five, side by side

	Memory	Aims at a goal	Weighs options	Improves itself
Simple reflex	no	no	no	no
Model based	yes	no	no	no
Goal based	yes	yes	no	no
Utility based	yes	yes	yes	no
L				

HOLD ON TO THIS

Every row is the row above plus one column. That is why it is a ladder and not a menu, and it is why you cannot have foresight without memory.

SECTION 02

PEAS, and the environment

The environment picks the architecture. Not your preference.

PEAS

Before you choose an architecture, describe the job

P	**Performance measure**	What counts as doing well? Not what it does, but how you would score it.
E	**Environment**	What can it sense and affect? For us, exactly the tool list and nothing else.
A	**Actuators**	How it changes the world. For us, the tool calls your loop executes.

WHY THIS IS NOT BUSYWORK

The performance measure is the one people skip, and it is the one that decides whether you need a utility function. If you cannot say what good looks like, you cannot build an agent that optimises for it, and you will not be able to tell whether the one you built is working.

Our hotel agent, described properly

P	A room is found, it is available, and it fits the person's budget and distance preference.
E	Three hotels, their rates, distances, ratings and per date availability. Nothing else exists.
A	get_room_availability, get_hotel_details, list_hotels. Three actuators, no more.
S	The strings those three tools return, plus everything said so far in the conversation.

READ THE E LINE AGAIN

The environment is exactly the tool list. An agent cannot perceive or affect anything you did not hand it. That sentence is a definition today and a security control in Unit 6.

Six properties, and what each one costs you

Fully or partially observable	Can it see everything relevant right now?	Partial forces internal state. Rung 2 minimum.
Deterministic or stochastic	Does the same action always give the same result?	Stochastic means you must handle surprise, not just plan.
Episodic or sequential	Does this action affect later ones?	Sequential forces foresight. Rung 3 minimum.
Static or dynamic	Can the world change while the agent is thinking?	Dynamic puts a clock on your reasoning.
Discrete or continuous	Finite set of states and actions, or not?	Continuous rules out simple enumeration.
Single or multi agent	Are others acting, possibly against you?	Multi agent is Unit 4, and it changes everything.

The third column is the useful one. The environment tells you the minimum rung you can get away with.

SECTION 03

Where the model sits

A fifty year old picture, and the new part inside it.

So which architecture did you build last week?

Forty lines. A goal, three tools, a loop, a budget. Put it on the ladder.

MOST PEOPLE SAY

Goal based

And they are right. It held a desired end state and worked out its own route to it. The step count was not fixed by us.

BUT LOOK CLOSER

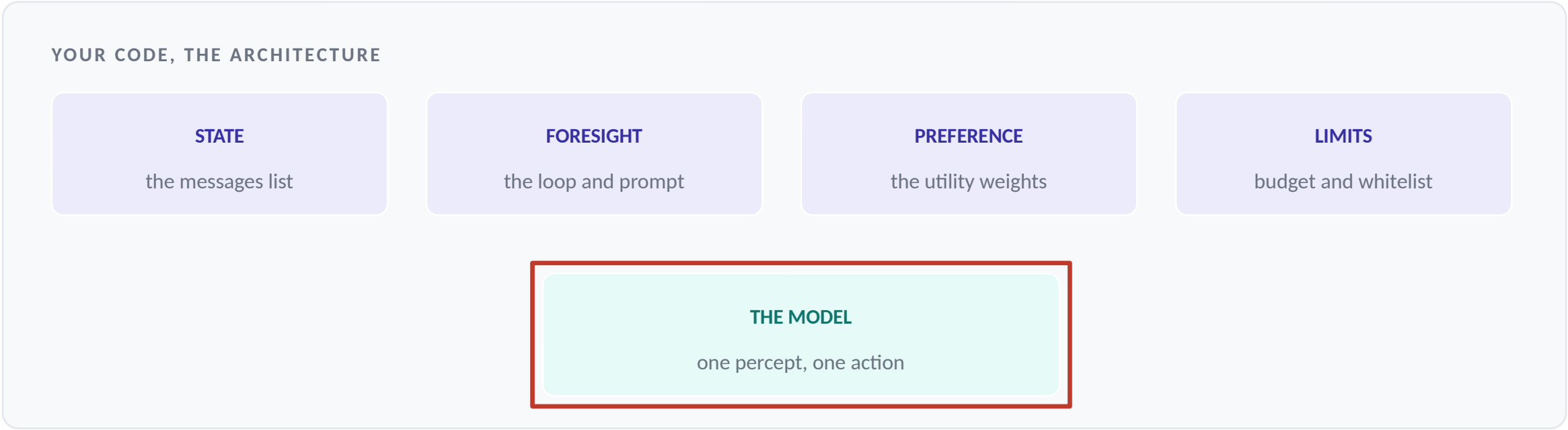
With a reflex core

Each individual turn is condition to action. Percept in, one action out. The foresight lives in the loop and the prompt, not in any single call.

THIS IS THE MOST USEFUL IDEA IN THE LECTURE

A language model is a very good reflex agent. One percept, one action, no memory. Everything above rung one, the state, the foresight, the preferences, is built around it by you. The architecture is your code. The model is a component inside it.

The same picture, drawn



a reflex agent, sitting inside a goal based one

Upgrading the model makes the red box better. It does not add a single one of the four boxes above it.

The misconception this kills

WHAT PEOPLE ASSUME

A better model means a better agent

So the fix for anything is always the newer, larger, more expensive model.

WHAT IS ACTUALLY TRUE

A better model means a better component

If your agent forgets things, has no plan, or cannot explain a trade off, those are architecture problems. No model upgrade fixes a missing messages list.

HOLD ON TO THIS

A useful diagnostic question when something misbehaves: is this the model choosing badly, or is this my architecture never giving it the chance to choose well?

SECTION 04

Live build

One task, four architectures, and you read the difference.

The experiment

One question, asked of four agents. The tools are identical. The loop is identical. Only the architecture changes.

> *"Find me a room for 14 August. I care about price most, then distance from campus."*

Three hotels are on file. The Taj Palace is full on that date, so the obvious first choice fails, which is deliberate. Watch which agents recover from that and which do not.

OPEN THIS NOW

notebooks/u1/l2_architectures.ipynb Run the setup cell. If it prints your lane you are ready. Each architecture is one cell after that.

Architecture one, reflex

```
REFLEX_SYSTEM = (
    "You are a lookup service. Answer using exactly one
     tool call, then stop. Never make a second tool call."
)

def reflex(client, model, request):
    return _loop(..., max_steps=2)
```

NOTICE HOW THE ARCHITECTURE IS ENFORCED

Not by trusting the prompt. By max_steps=2, which makes a second round structurally impossible. A prompt is a request. A budget is a guarantee. When the two disagree, only one of them is load bearing.

HOLD ON TO THIS

Run it and it will tell you the Taj Palace is full, and then stop. Correct behaviour, useless answer. That gap is why rung one exists on the ladder rather than in production.

Architecture two, model based

```
class ModelBasedAgent:
    def __init__(self, client, model):
        self.messages = [{"role": "system", ...}]

    def ask(self, request):
        self.messages.append({"role": "user", ...})
        return _loop(..., self.messages, max_steps=4)
```

WHY THIS IS A CLASS AND THE OTHERS ARE FUNCTIONS

The internal state is the architecture. Moving messages from a local variable into an attribute is the entire difference between rung one and rung two, and it is four characters of Python. Try "what about the fifteenth" as a second question and watch it resolve.

Architecture three, goal based

```
GOAL_SYSTEM = (
    "You are given a goal, not a set of steps. Work out
     for yourself which tools to call and in what order.
     Keep going until the goal is satisfied."
)

def goal_based(client, model, goal, max_steps=8):
    return _loop(..., max_steps=max_steps)
```

Same tools. Same loop. The budget went from 2 to 8, and the prompt stopped dictating the shape of the answer.

WATCH THE TRACE

It lists the hotels, finds the Taj Palace full, and moves on by itself. Nothing in your code said "if full, try another". That recovery is what foresight buys you, and it is the first architecture on the ladder that is useful here.

Architecture four, utility based

```
utility_based(client, MODEL,
    goal="Find me a room on 2026-08-14",
    preferences={"price": 0.5,
                 "distance from campus": 0.3,
                 "rating": 0.2})
```

The weights are the utility function. Written down, passed in, and therefore arguable. Change 0.5 to 0.1 and the recommendation changes in front of the room.

THE OUTPUT IS DIFFERENT IN KIND, NOT JUST QUALITY

The goal based agent returns an answer. This one returns a ranked comparison and names the trade off it made. In any setting where somebody has to justify the decision later, that difference is the entire product.

Read the four traces together

REFLEX	**1 call**	Taj Palace is full.	Correct. Useless.
MODEL BASED	**2 to 4 calls**	Taj Palace is full. Remembers if you ask again.	**Better conversation, same dead end.**
GOAL BASED	**4 to 6 calls**	Recovers on its own, finds a hotel with rooms.	**First one that solves the task.**
UTILITY BASED	**6 to 9 calls**	Compares all viable hotels, scores them, recommends.	**Solves it and justifies it.**

THE COST, WHICH THE TABLE ALSO SHOWS

Going up the ladder multiplied the API calls by roughly six. Every rung is bought with money and latency. Choosing rung four when rung two would do is not sophistication, it is waste, and Unit 2 makes you defend the choice.

Hold on to this

The model is a reflex agent. Every rung above that, you build.

Each rung adds exactly one thing	State, then foresight, then preference, then improvement.
The environment picks the rung	Partially observable forces state. Sequential forces foresight.
Memory is not learning	Almost nothing you build this semester is a learning agent.
Every rung costs money	Utility based used roughly six times the calls of reflex.

Before next session

DO	**Practical 2**	The hotel information agent. Multi turn, a third tool of your own, honest failure, and a termination guard you can justify.
DO	**The weight experiment**	Run utility_based three times with different weights. Record the recommendation each time. Two cells, both with outputs.
THINK	**PEAS one system**	Pick any system you use daily. Write its PEAS in four lines. Then say which rung it needs and why.

ON THE SECOND ONE

If all three weightings give the same recommendation, your weights are not doing anything and you should say so rather than hide it. A utility function that never changes the answer is decoration.

Next session

Unit 1, Lecture 3

Conversational AI Fundamentals

You have built agents that answer once and agents that hold a conversation. Next we take the conversation itself seriously. Turns, context windows, what actually gets sent on every request, and why a long chat quietly gets more expensive and less accurate at the same time.

COME WITH

Practical 2 submitted, your weight experiment committed, and one PEAS description.